

4

Applying Clean Architecture to an ASP.NET Core Solution

With the previous chapter still fresh in your memory, this is the chapter where you will apply clean architecture to an ASP.NET Core 5 solution. This chapter teaches you the real-world organization of files, folders, projects, and ASP.NET Core app dependencies, preparing you for future big and scalable ASP.NET Core 5 enterprise applications.

Organizing your code, files, and folders helps other developers to understand your code, refactor it, and add future features, something that happens most of the time in a real-world application.

We will cover the following topics:

- Introducing clean architecture
- The core layer
- The infrastructure layer

- The presentation layer
- Managing the `Tests` folder
- Structuring a clean architecture solution

Technical requirements

Here is what you need to complete this chapter:

- **For building a backend:** Visual Studio 2019, Visual Studio for Mac, or Rider
- **For scaffolding a project:** The .NET Core CLI
- Bash terminal for Mac and Linux users
- PowerShell or the Git Bash terminal for Windows users

Here is the link to this chapter's finished repository: <https://github.com/PacktPublishing/ASP.NET-Core-and-Vue.js/tree/master/Chapter04>.

Introducing clean architecture

So, what on earth is clean architecture? In a nutshell, **clean architecture** is a principle of organizing software architecture to keep developers away from the difficult refactoring of the application in the future. Clean architecture helps you build a service for a specific domain model that prepares it for microservice architecture.

Here is a diagram of clean architecture in ASP.NET applications:

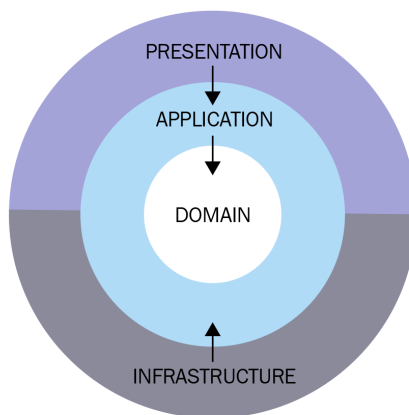


Figure 4.1 – Clean architecture diagram

In clean architecture, two layers must be the core or center of the structure. These layers are the domain layer, which contains most of your entities, enums, and settings. The application layer keeps most of your **Data Transfer Objects (DTOs)**, interfaces, mappings, exceptions, behaviors, and business logic.

The difference is that the enterprise logic could be shared across many systems, whereas the application logic or business logic is specific. Now, rather than having the core dependent on data access and infrastructure, we invert those dependencies. So, presentation and infrastructure depend on the core, but the core has no dependency on either layer. This structure is done by adding abstractions or interfaces within the application layer, which are implemented outside of the application layer in other layers. For instance, if we wanted to implement the repository pattern, we would add an `IRepository` interface within the infrastructure application and implementation.

With this design principle, all dependencies point toward the middle. Hence, you can see that the core has no dependencies on other layers. Subsequently, presentation and infrastructure depend on the core, but not on one another. This is very important because we want to make sure that the logic we create for this system stays within the core. For example, if the presentation took a dependency on infrastructure to send some notifications, it is a logic. This logic now has to appear within the presentation layer. Hence it has to orchestrate the interaction between presentation and infrastructure. Typically, we don't want that to happen because we can't reuse that logic. We want a system that's going to last for a decade at least.

Here is the thing. If we have a frontend web app with a backend application, there is a high probability that it is not going to be there in 10 or 15 years. We need the logic inside the core, where it is isolated from all of those things.

If the circular diagram (see *Figure 4.1*) of the clean architecture is a bit difficult to understand, we can look at the following illustration:

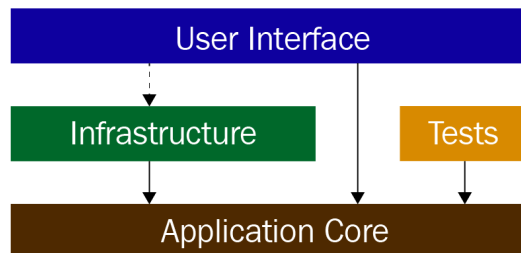


Figure 4.2 – A flat diagram of clean architecture

The preceding diagram is a simplified and flattened version of the clean architecture.

This diagram illustrates that your application core is at the very bottom, which is the center of the clean architecture circle diagram (see *Figure 4.1*).

As you can see, the app core has no dependencies. On the other hand, the UI or the presentation layer is dependent on the application core. You will also notice that the project infrastructure is dependent on the app core.

In the end, the whole application becomes highly testable. You can quickly write unit tests, integration tests, and functional tests.

Now let's break these down into more digestible content. We'll separate the layers into three folders with .NET projects inside:

- The `core` folder will have `Application` and `Domain` projects.
- The `infrastructure` folder will have `Data` and `Shared` projects.
- The `presentation` folder will have a `WebApi` project.

This is the setup of the projects and folder arrangement of the ASP.NET Core 5 solution.

The core layer – directory

The **core layer** is the center of clean architecture. We will implement this as a directory or folder inside a Visual Studio solution that can be created using the .NET Core CLI. All other project dependencies must point toward the core to the domain and application project to be specific. Similarly, the core layer will never depend on any other layers.

To set up the core layer, we need two projects – the `Domain` and `Application` projects.

Domain – project

This clean architecture part is a .NET Standard 2.1 class library project with entities, interfaces, enums, DTOs, and so on.

The `Domain` project must have an empty project reference, which shows that it does not have any dependencies on any project.

Application – project

This part of the clean architecture is also a .NET Standard 2.1 class library project. It has defined interfaces, but the implementations are outside of this layer. This project also has the command and queries of the CQRS pattern, the behaviors of MediatR, the profile of AutoMapper, exceptions, models, and so on.

We will create the aforementioned projects in the *Structuring a clean architecture solution* section of this chapter.

What if you just started to build microservices and realized that there is code in the core layer you want to reuse in other services or domains?

Shared Kernel – NuGet project

Ideally, the `Shared Kernel` project is a private NuGet package for sharing code between multiple projects, in microservices, for instance. This project helps us easily swap out which version we depend on without breaking the work of our peers or other teams building other solutions.

Now let's take a look at the infrastructure layer.

The infrastructure layer – directory

The **infrastructure layer** has the class implementations of interfaces defined in the `Application` project. Resources such as SMTP, filesystems, or web services are samples of the application's external dependencies but implemented in this layer.

This layer is another directory inside the solution while holding multiple projects. The projects we will add here are for data and shared projects. We can also add a project named `Identity` for authentication, but we will do this in *Chapter 9, Securing ASP.NET Core*, to keep the structuring part of this clean architecture minimal. So here they are.

To set up the infrastructure layer, we will need two projects – `Data` and `Shared`.

Data – project

This part of the infrastructure layer is a .NET 5.0 class library project intended for a database. You can also name this data project to a persistence project; persistence and data are intractable.

Shared – project

This part of the infrastructure layer is also a .NET 5.0 class library project. This project will include shared code between different services, such as Email, SMS, or Date Time.

Again, you will create the infrastructure layer projects in the *Structuring a clean architecture solution* section of this chapter. In the meantime, let's move to the next section, which is the presentational layer.

The presentation layer – directory

The **presentation layer** is where you build your web application. The web application can use ASP.NET Core MVC, ASP.NET Core Web API, a **Single-Page Application (SPA)**, or a mobile application.

The real-world application that we will be building throughout the chapters of this book requires a Web API project and a website.

WebApi – project

WebApi is an ASP.NET Web API project that uses .NET 5.0. This project interacts with any client-side applications, such as on the web, mobile, desktop, and **Internet of Things (IoT)**.

Also, WebApi depends on both the Application and infrastructure layers.

client-app – non-project web application

client-app is the web application that serves as the user interface of the application. We will create the Vue.js 3 application in *Chapter 11, Vue.js Fundamentals in a Todo App*, and it will be inside the `presentational` folder. Name it `client-app`.

That's it for the presentation layer. So, where do you think we should place the tests for the application? Let's move on to the next section.

Managing tests – directories

This section is not part of the clean architecture principle, but the best practice is to use separation of concerns. Hence, organizing the test projects based on the tests they do, such as unit tests, functional tests, integration tests, and load testing, is the best practice.

I will lay out the test projects in the following subsections, but we will not create them in this chapter yet. The following are the test projects.

Unit test – project

A **Unit test** project tests small and specific parts of your code. This project can be created using an XUnit, NUnit, or MSTest project.

Integration test – project

An **Integration test** project tests whether the components are working together. This project can be created using an XUnit, NUnit, or MSTest project.

Now that we've finished the part where developers write their tests in ASP.NET Core, it is time to create the layers and projects for an ASP.NET Core 5 solution.

Structuring a clean architecture solution

This section is that part where we will create a solution, directories, and projects to build a clean architecture solution for the application. The application is about travel lists and places worldwide where an admin can add new, delete, update, and read places from the travel destination list.

Before we begin, open up your terminal and navigate to your desktop. Make sure your terminal or command line is in the `Desktop` directory.

For Windows users, use PowerShell or the Git Bash terminal. The Git Bash terminal is part of the Git version control installer you used in *Chapter 2, Setting Up a Development Environment*. If you are going to use PowerShell, remember to use a backslash instead of a forward slash:

1. Let's start with the folder of the solution by running the following command:

```
mkdir Travel
```

This command creates a folder named `Travel`.

2. The next step is to go inside the `Travel` folder:

```
cd Travel
```

Your terminal should be inside the `Travel` directory by now.

3. Now, use the `dotnet` CLI:

```
dotnet new sln
```

This command creates a solution file and gets its name from the folder where it is located.

4. The next step is to create a folder named `src`:

```
mkdir src
```

The `src` directory is inside the `Travel` directory.

5. Now, navigate to the `src` directory or folder:

```
cd src
```

While inside the `src` directory, we will create three folders here, the three layers, namely – `core`, `infrastructure`, and `presentation`.

6. Now let's create the `core` folder:

```
mkdir core
```

This command creates the `core` directory.

7. Now create the folder for `infrastructure`:

```
mkdir infrastructure
```

This command creates the `infrastructure` directory or folder.

8. And now, create the `presentation` directory or folder:

```
mkdir presentation
```

This command creates the `presentation` directory or folder.

9. Go inside the `core` directory:

```
cd core
```

While inside the `core` directory, we will create two projects. The first project will be for the `Domain` layer:

```
dotnet new classlib -f netstandard2.1 --name Travel.  
Domain
```

This command creates a new .NET Standard 2.1 class library named `Travel . Domain`. The second project will be for the `Application` layer:

```
dotnet new classlib -f netstandard2.1 --name Travel.  
Application
```

This command creates a new .NET Standard 2.1 class library named `Travel.Application`.

10. Then, go inside the `Travel.Application` directory:

```
cd Travel.Application
```


This command navigates you to the `Travel.Application` folder.

11. Then we need to add a reference to the `Travel.Domain` project:

```
dotnet add reference ../Travel.Domain/Travel.Domain.csproj
```

Now, the `Travel.Application` project has a dependency on the `Travel.Domain` project, but only this project.

12. Now, navigate to the `infrastructure` directory:

```
cd ../../infrastructure
```

While inside the `infrastructure` directory, we will create the projects for `Data` and `Shared`.

13. Let's now create the infrastructure for `Data` or persistence:

```
dotnet new classlib -f net5.0 --name Travel.Data
```

This command creates a new .NET 5.0 class library project named `Travel.Data`.

14. Let's now create the infrastructure for `Shared`:

```
dotnet new classlib -f net5.0 --name Travel.Shared
```

This command creates another .NET 5.0 class library project named `Travel.Shared`.

15. Then, go inside the `Travel.Data` directory:

```
cd Travel.Data
```

While inside the `Travel.Data` directory, we need to make `Travel.Data` dependent on the `Domain` and `Application` layers.

16. Let's add a reference to the `Travel.Domain` project first:

```
dotnet add reference ../../core/Travel.Domain/Travel.Domain.csproj
```

Now, `Travel.Data` is dependent on the `Travel.Domain` project.

17. Now, let's add a reference to the `Travel.Application` project:

```
dotnet add reference ../../core/Travel.Application/Travel.Application.csproj
```

This command creates a dependency on `Travel.Data` to `Travel.Application`.

18. Now, go to the `Travel.Shared` project:

```
cd ../Travel.Shared
```

This command takes you to the directory of the `Travel.Shared` project.

19. Now we can add a reference to the `Travel.Application` project:

```
dotnet add reference ../../core/Travel.Application/  
Travel.Application.csproj
```

This command creates a dependency on the `Travel.Shared` project to the `Travel.Application` project.

20. Now, let's go to the presentation layer:

```
cd ../../presentation
```

Now that we are in the presentation layer, we will create a Web API project here.

21. We will create the latest ASP.NET Core 5 webapi here:

```
dotnet new webapi --name Travel.WebApi
```

This command creates a new Web API project named `Travel.WebApi`.

22. Let's navigate inside the `Travel.WebApi` folder:

```
cd Travel.WebApi
```

Now, while inside `Travel.WebApi`, we will add some references here.

23. Let's add a reference to the `Application` layer:

```
dotnet add reference ../../core/Travel.Application/  
Travel.Application.csproj
```

This command adds a dependency to the `Travel.Application` project.

24. Now let's add a reference to `Data` for persistence:

```
dotnet add reference ../../infrastructure/Travel.Data/  
Travel.Data.csproj
```

This command adds a dependency to the `Travel.Data` project.

25. The next step is to add a reference to the Shared project:

```
dotnet add reference ../../infrastructure/Travel.Shared/  
Travel.Shared.csproj
```

This command adds a dependency to the Travel.Shared project.

26. Now we can go back to the root folder of the application solution:

```
cd ../../../../
```

Since the solution file is in the root folder, we can register all the projects to the solution file.

27. The first project we will add to the solution file is the Domain layer:

```
dotnet sln add src/core/Travel.Domain/Travel.Domain.  
csproj
```

This command lets the solution file know the Travel.Domain project.

28. We will also add the Application layer of the app:

```
dotnet sln add src/core/Travel.Application/Travel.  
Application.csproj
```

This command lets the solution file know the Travel.Application project.

29. The next project to add is Data for persistence:

```
dotnet sln add src/infrastructure/Travel.Data/Travel.  
Data.csproj
```

This command lets the solution file know the Travel.Data project.

30. We also have to add the Shared project:

```
dotnet sln add src/infrastructure/Travel.Shared/Travel.  
Shared.csproj
```

This command lets the solution file know the Travel.Shared project.

31. Now, the last project to add is the Web API project:

```
dotnet sln add src/presentation/Travel.WebApi/Travel.  
WebApi.csproj
```

This command lets the solution file know the Travel.WebApi project.

You have now finished structuring an ASP.NET Core 5 application with a clean architecture. Close down your terminal and double-click the solution file to open up the whole application.

Let's see the application in either Visual Studio 2019, Visual Studio for Mac, or Rider.

Visual Studio 2019

In Visual Studio 2019, this is how it should look:

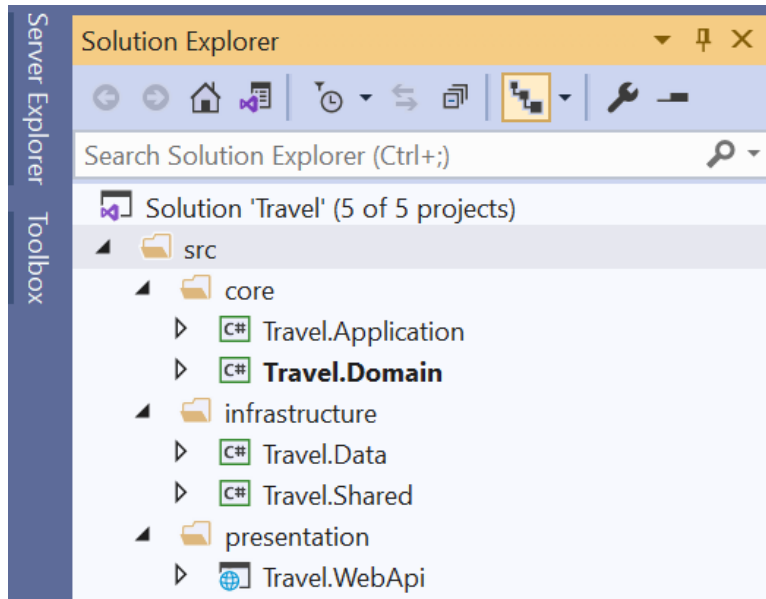


Figure 4.3 – Folder structure in Visual Studio 2019

Visual Studio 2019 shows **Travel** as the solution that has the `src` directory. The `core`, `infrastructure`, and `presentation` layers must be under the `src` directory. You will also see that the `Application` and `Domain` projects are under the `core` directory, while the `Data` and `Shared` projects are under the `infrastructure` directory.

Similarly, you can find the ASP.NET Core 5 WebApi under the `presentation` directory.

Now let's see the application for Mac users.

Visual Studio for Mac

In Visual Studio for Mac, this is how it should look if you followed along correctly:

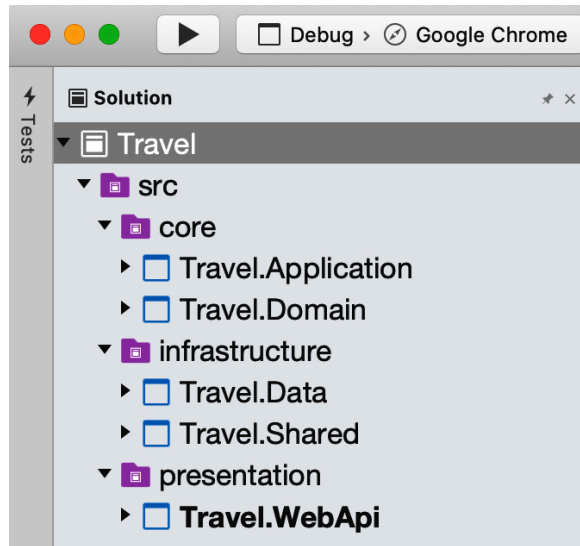


Figure 4.4 – Folder structure in Visual Studio for Mac

`Travel` is the name of the application solution, and below that is the `src` directory, which has the `core` layer, `infrastructure` layer, and `presentation` layer in the clean architecture.

Moreover, the `Application` and `Domain` projects are under the `core` directory, while the `Data` project and the `Shared` project are under the `infrastructure` directory. Finally, the ASP.NET Core 5 `WebApi` project must be inside the `presentation` directory.

That's all for Mac users. The structure should be the same as Visual Studio 2019. Likewise, the structure should be the same as the next section, which is for users of Rider. Now let's see how the Rider IDE shows the application solution.

Rider

Lastly, if you are using Rider, the following screenshot shows you the structure of your application solution:

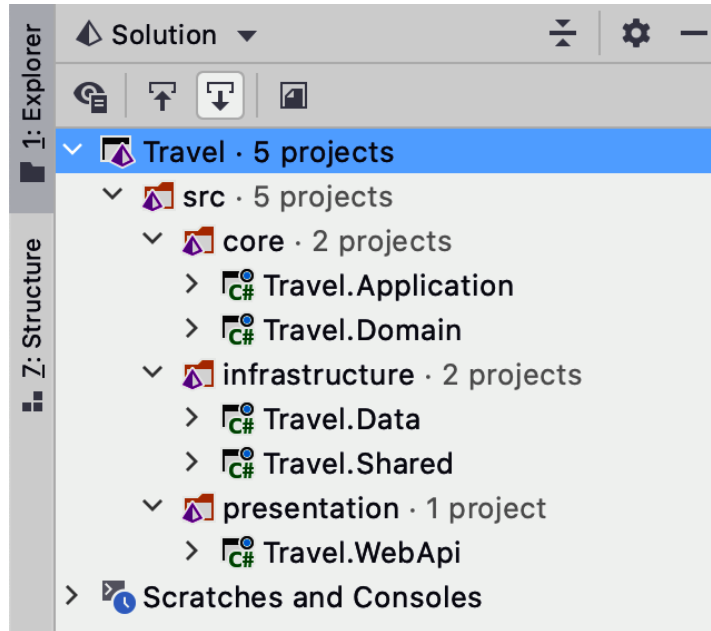


Figure 4.5 – Folder structure in Rider

Again, the screenshot displays `Travel` as the solution's name, and it has the `src` directory, which holds the `core` layer, `infrastructure` layer, and `presentation` layer.

The `Application` and `Domain` projects are inside the `core` directory, while the `Data` and `Shared` projects are inside the `infrastructure` directory.

In the `presentation` folder, you should see the ASP.NET Core `WebApi` project

OK, so that was a well-layered and structured clean architecture solution. Here is the GitHub repository of the source code after finishing this chapter:

<https://github.com/PacktPublishing/ASP.NET-Core-5-and-Vue.js-3/tree/master/Chapter-4/Finish/Travel>.

Now let's summarize everything we have learned here.

Summary

With that, we have learned what clean architecture is and how it will help developers build applications that can last for a decade or more. We also learned that clean architecture makes our service testable and ready for microservices.

We've tackled what comprises clean architecture. There is the core layer that does not have any dependencies on the infrastructure and presentation layers. An infrastructure layer communicates with external sources and the presentation layer, which your users use and interact with.

You've also learned how to structure tests in clean architecture and lastly, we learned how to use the `dotnet` CLI to build an ASP.NET Core solution and project.

Now that you've improved your skills by learning how to apply clean architecture to an ASP.NET Core 5, this will help you to build a highly scalable and testable application in the future.

In the next chapter, we will set up our database and build routing and controllers to see how they go about processing HTTP requests.

